

Distributed Computing: Spark and Managed Platforms

Databricks, Snowflake, and the Convergence of Cloud Analytics

Phase 3: Distributed Computing



Project 2: Team and Idea identification due this week

Distributed Computing Intro Recap: What You Learned

Why Distributed

The **clock speed wall**,
data wall,
and
cost wall
force us to spread work across
machines

How It Works

MapReduce: Map (parallel) →
Shuffle (redistribute by key) →
Reduce (aggregate)

What's Hard

- Partitioning
- Fault tolerance
- CAP theorem choices
 - CP vs AP
- Coordination overhead

Managed Services: When someone solves these problems for you

This Week's Shift: Local Compute → Cloud Compute

Week 9: Cloud Data, Local Compute

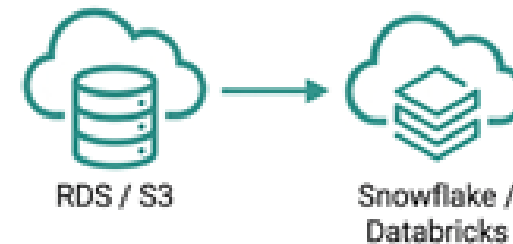
- Queried RDS PostgreSQL via DBeaver and psycopg2
- Downloaded data to your laptop
- Analyzed locally with pandas and scikit-learn
- **Binding constraint: your laptop's memory and processing power**



Week 9: Cloud Data, Local Compute

Week 10: Cloud Data, Cloud Compute

- Data stays in cloud storage
- Processing happens on Snowflake or Databricks
- Your laptop is just the interface (browser)
- **Binding constraint: platform configuration and cost**



Week 10: Cloud Data, Cloud Compute

Same data, same analytical questions, but the compute moves to the cloud.

The Binding Constraint Shifts

The binding constraint no longer compute. It's operational overhead.

Self-Hosted PostgreSQL

→ **RDS**

(Week 7)

Manual ETL Scripts

→ **Glue**

(Week 6)

Local Python Analysis

→ **Databricks / Snowflake**

(This week)

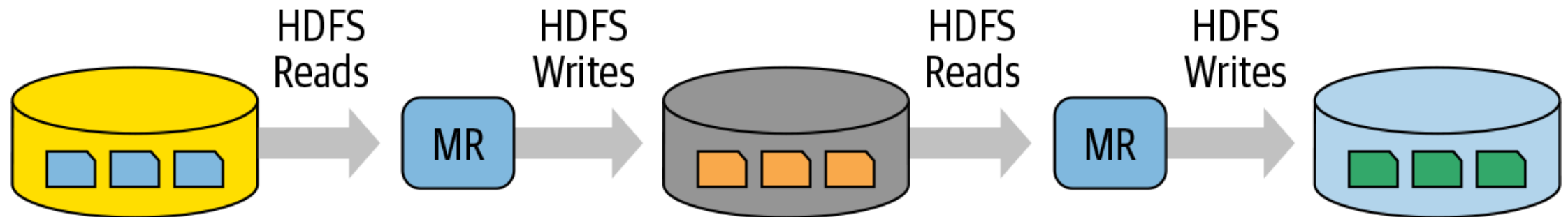
Same engine (or equivalent), different operational model

Organizations pay for managed platforms not because the technology is better, but because managing it yourself is its own full-time job.

The Spark Engine

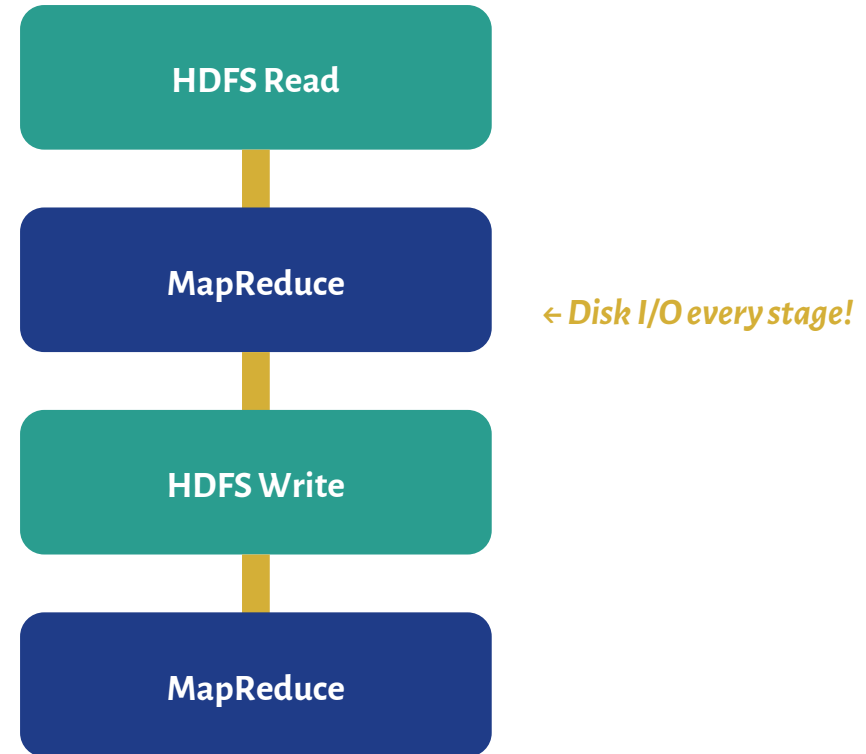
What Powers Databricks, EMR, Glue, and the Modern Data Stack

Back and Forth with MapReduce



The Problem Spark Solved

- Hadoop MapReduce was groundbreaking (Week 9)
- But every MR stage wrote intermediate results to disk
- Multi-stage jobs = repeated disk I/O
- Disk is 100× slower than memory
- Network is 1,000,000× slower than memory



Spark's insight: keep data in memory between stages instead of writing to disk.

Latency Numbers Every Programmer Should Know

Operation	Latency	Human Scale
L1 cache reference	0.5 ns	< 1 μ s
Branch mispredict	5 ns	< 1 μ s
L2 cache reference	7 ns	< 1 μ s
Mutex lock / unlock	25 ns	< 1 μ s
Main memory reference	100 ns	< 1 μ s
Compress 1K bytes (Zippy)	3,000 ns	= 3 μ s
Send 2K over 1 Gbps network	20,000 ns	= 20 μ s
SSD random read	150,000 ns	= 150 μ s
Read 1 MB from memory	250,000 ns	= 250 μ s
Roundtrip same datacenter	500,000 ns	= 0.5 ms
Read 1 MB from SSD	1,000,000 ns	= 1 ms
Disk seek	10,000,000 ns	= 10 ms
Read 1 MB from disk	20,000,000 ns	= 20 ms
Send packet US $\rightarrow$ Europe $\rightarrow$ US	150,000,000 ns	= 150 ms

What Is Apache Spark?



A distributed compute engine that keeps data immutable and in memory, enabling batch, interactive, and streaming workloads.

Immutable Data

- Data structures are never modified in place. Transformations create new structures.
- The underlying abstraction is the RDD, but you'll work with DataFrames, the modern, schema-aware API.
- Same immutability as Parquet (Week 6).

In-Memory Processing

- Intermediate results stay in RAM between stages, avoiding Hadoop's disk I/O penalty.
- If data exceeds memory, Spark spills to disk (slower, but still works).

Lazy Evaluation

- Operations are recorded in a plan (DAG) and only executed when results are needed.
- This delay lets the optimizer see the whole picture before running anything.
- Deep dive in Week 11.

Origins and Design Philosophy

- 2009: Started at UC Berkeley's AMP Lab by researchers who used Hadoop and understood its limits
- 2013: Donated to Apache Software Foundation as open-source
- 2014: Founders launched Databricks, a commercial platform built around their own engine

Speed

In-memory processing + DAG optimizer

Ease of Use

Python, SQL, Scala, R, Java APIs (not raw MapReduce)

Modularity

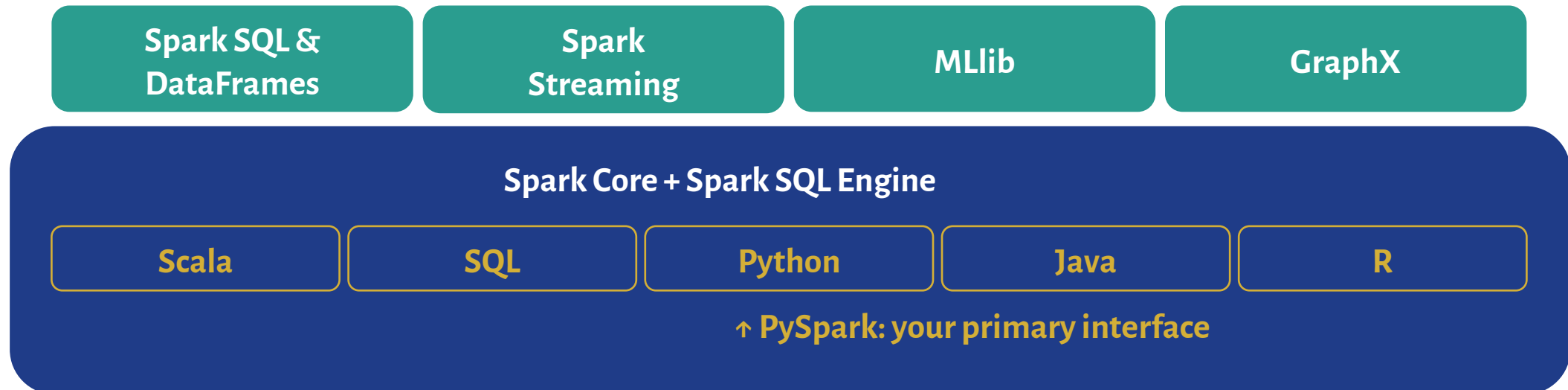
SQL, ML, Streaming, Graph all in one engine

Extensibility

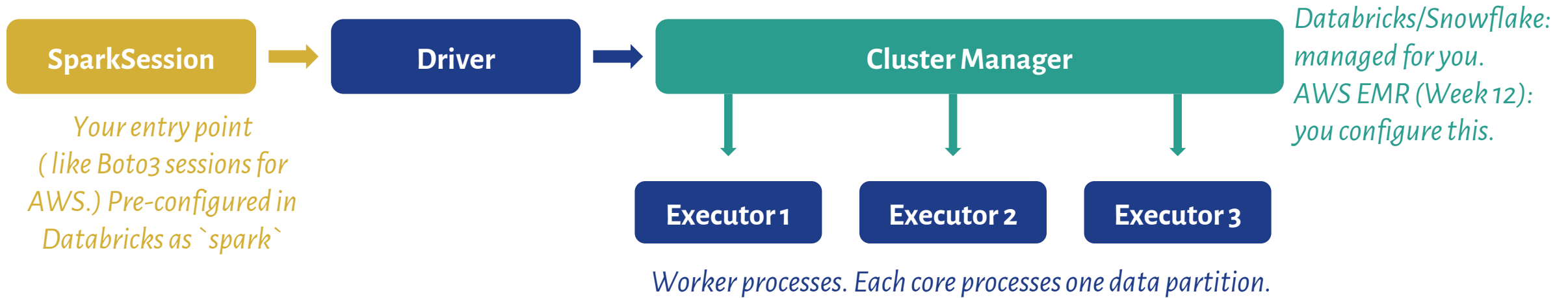
Connectors for hundreds of data sources

The founders didn't build a competing technology. They built a product around the engine they'd already created.

The Unified Spark Stack



Spark Architecture: SparkSession → Driver → Executors



Databricks Architecture

Building a Product Around the Engine

What Is Databricks?

A managed analytics platform built on Apache Spark, adding collaborative notebooks, governed data storage (Delta Lake), a metadata catalog (Unity Catalog), and infrastructure management.

- Founded 2014 by Spark's creators at UC Berkeley
- Runs on AWS, Azure, or GCP (Databricks doesn't own data centers)
- Your data stays in your cloud account (important for governance)
- Pioneered several innovations that are now industry-standard

When Databricks launched, multi-language notebooks, lakehouse architecture, and unified governance catalogs were genuinely new. By 2026, competitors (especially Snowflake) have adopted equivalents for most of them. Understanding what was innovative and why helps you evaluate any platform.

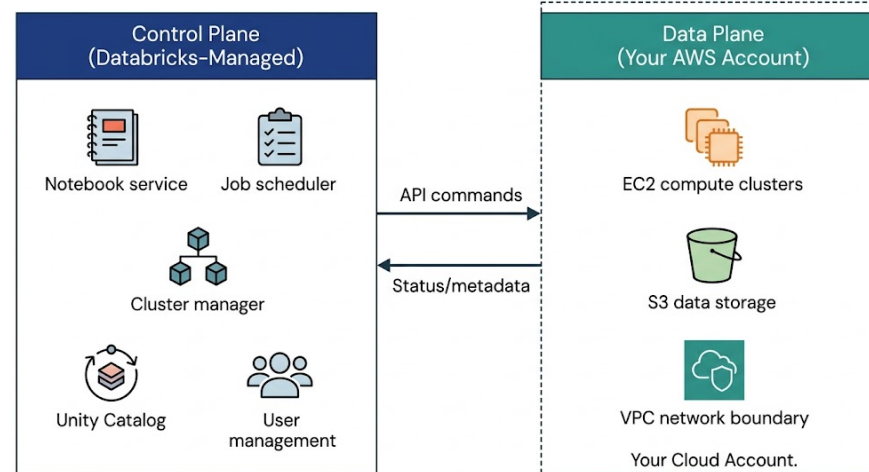
Control Plane vs. Data Plane

Control Plane — Managed by Databricks

- Notebook service and web UI
- Job scheduler
- Cluster manager
- Unity Catalog metadata
- User management & SSO

Data Plane — Runs in YOUR Cloud Account

- Compute clusters (EC2 instances)
- Data storage (S3 buckets)
- Network infrastructure (your VPC)
- **Your data never leaves your account**



Week 8 callback: this separation implements the data sovereignty principle.

The Data Lakehouse

Data Warehouse

- Structured, schema-on-write
- SQL-friendly, governed
- Expensive to store everything

Redshift, Snowflake



Data Lake

- Raw files, schema-on-read
- Flexible, cheap storage (S3)
- No governance by default

S3 + Glue, HDFS



Data Lakehouse

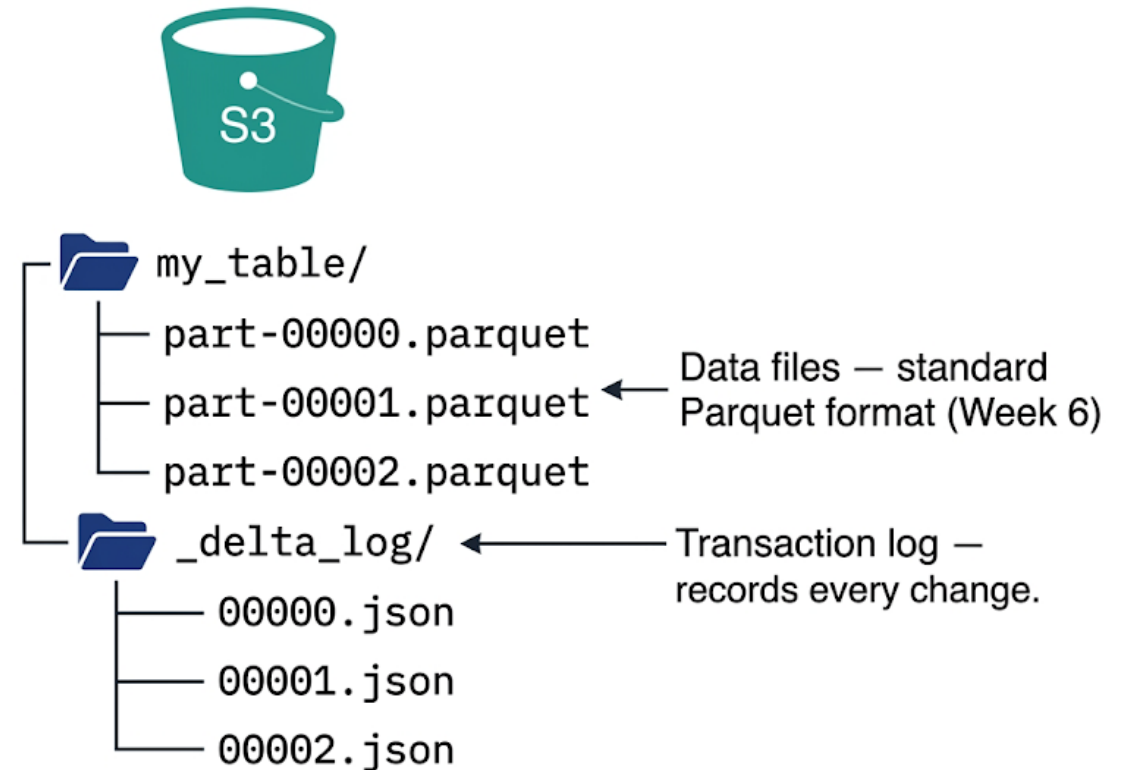
Open file formats on cheap storage + ACID transactions + governance + SQL performance

Powered by: Delta Lake (Databricks) or Apache Iceberg (industry open standard)

You've used tools from both worlds — Snowflake/Redshift (warehouse) and S3/Glue (lake). The lakehouse combines both.

Delta Lake: Parquet + Transaction Log

- Parquet files are immutable
- Transaction log tracks which files are 'current'
- Enables: UPDATE, DELETE, MERGE, time travel, rollback
- Apache Iceberg uses the same pattern with a different log format



What the Transaction Log Makes Possible

ACID Transactions

Multiple operations succeed or fail together.
No partial writes, no corrupted tables.

Time Travel

Query any past version: `SELECT * FROM my_table VERSION AS OF 5.`
Audit, rollback, reproducibility.

Schema Enforcement

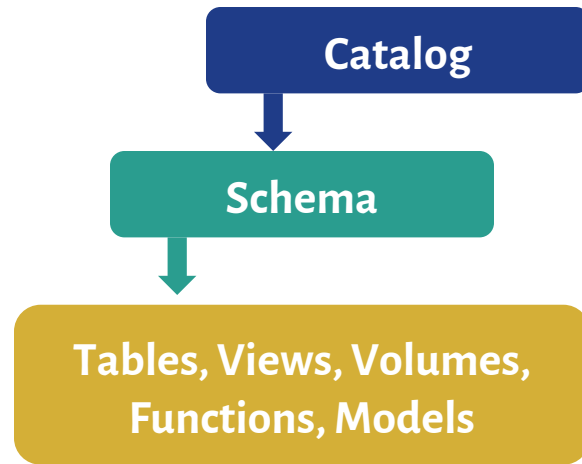
New data must match the table's schema.
Prevents accidental corruption.

MERGE / Upsert

`MERGE INTO target USING source ON key`
`WHEN MATCHED`
`THEN UPDATE`
`WHEN NOT MATCHED`
`THEN INSERT.`
Impossible on plain Parquet.

Snowflake offers equivalent capabilities: time travel, fail-safe, schema enforcement, via its own internal metadata system.

Unity Catalog: Three-Level Namespace



Example:

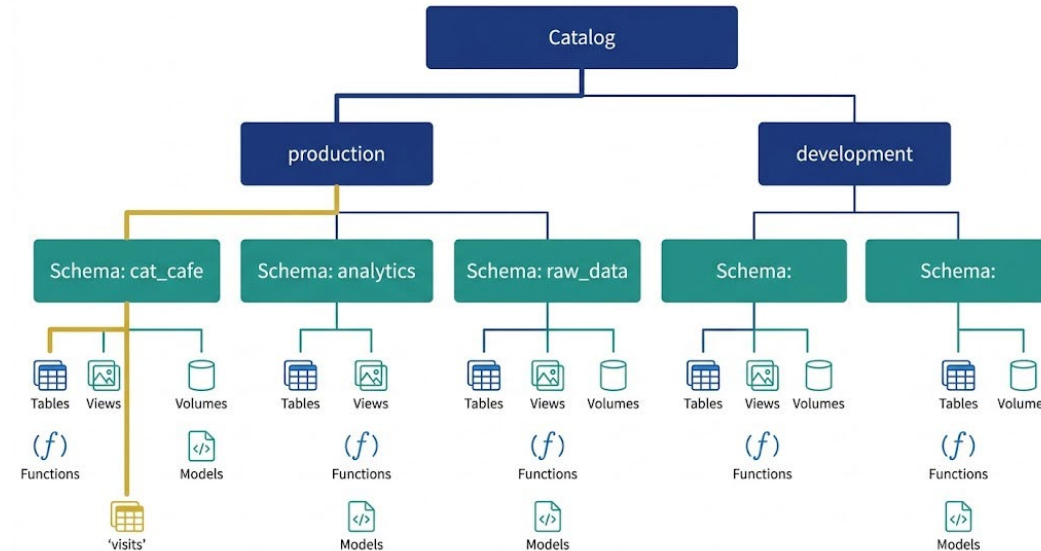
`production.cat_cafe.visits`

Governance Features

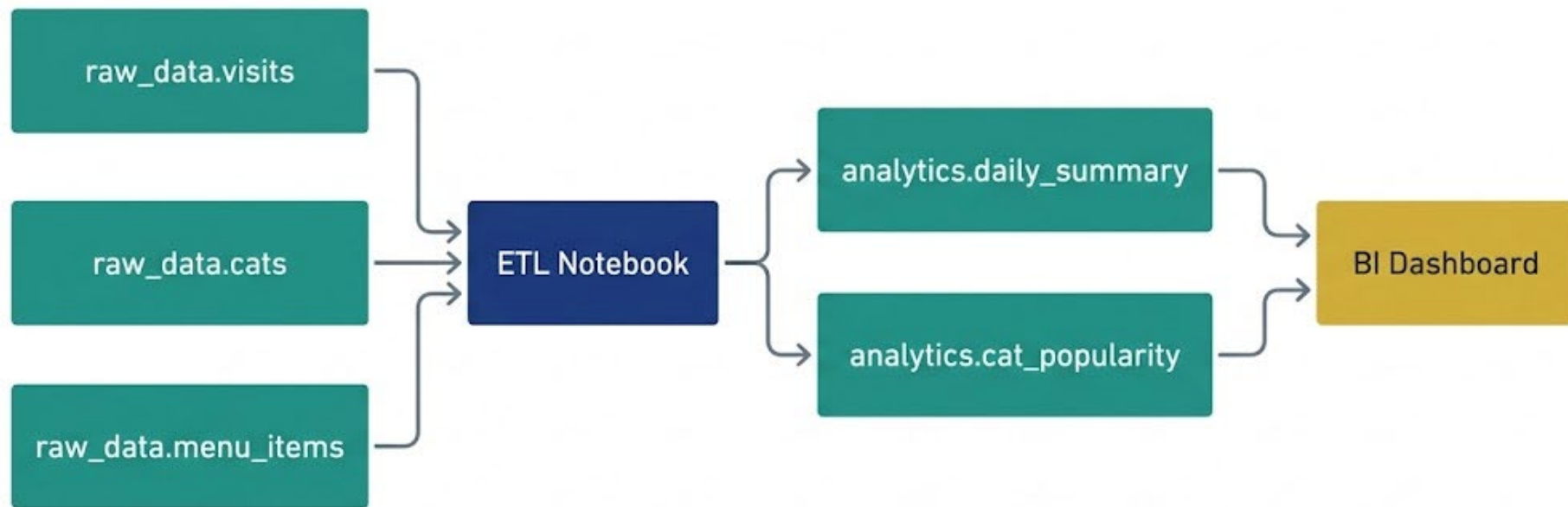
- Fine-grained access control (GRANT / REVOKE)
- Column-level and row-level security
- Data lineage tracking
- Audit logging

Implements RBAC and least-privilege from Week 8

Snowflake equivalent: Snowflake Horizon



Data Lineage: Where Did This Table Come From?



Lineage tracking answers: ***"If I change this source table, what breaks downstream?"***

Available in both Databricks (Unity Catalog) and Snowflake (Horizon)

The Databricks Stack

Metadata &
policies
(control plane)

Governance (Unity Catalog)

Processing
(cluster in your
VPC)

Platform Services (Notebooks, Jobs, SQL Warehouse)

Compute Engine (Apache Spark + Photon)

Your data
(your cloud
account)

Open Formats (Parquet / Delta Lake / Iceberg)

Cloud Storage (S3)

Every layer maps to something you've already learned this semester.

DISCUSSION

*Spark is open-source and free.
Snowflake and Databricks cost real money. What are you actually paying for?
When would a government agency choose one over the other — or both?*

- What operational tasks does each platform eliminate?
- What capabilities exist in one but not the other?
- How would you justify the cost to a budget officer?

Platform Convergence

What Databricks Pioneered, What's Now Standard, and What Still Differs

A Common Pattern in Cloud Computing

One company innovates.

Others become fast followers.

Capabilities converge.

Switching costs remain high.

- AWS pioneered cloud IaaS → Azure, GCP followed
- Databricks pioneered the lakehouse → Snowflake adopted open table formats
- OpenAI pioneered consumer LLMs → Anthropic, Google, Meta followed

This should make you confident: the skills you learn on one platform transfer to others.

Innovation → Industry Standard

Innovation	Databricks	Snowflake
Multi-language notebooks	Python, SQL, Scala, R	SQL, Python
Lakehouse / open formats	Delta Lake (2019)	Iceberg support
Governance catalog	Unity Catalog	Snowflake Horizon
DataFrame API	PySpark DataFrames	Snowpark DataFrames
AI/ML integration	MLflow, Mosaic AI	Cortex AI, Snowpark ML

Databricks pioneered. Snowflake adopted. Both platforms are converging.

What Still Differs: Architectural DNA

Databricks

- Built on Apache Spark (open-source engine)
- Data plane in YOUR cloud account
- You see EC2 instances, S3 buckets
- More control, more configuration
- Flexible: Python-first, notebooks + jobs
- Stronger for: data engineering, ML, streaming

Snowflake

- Proprietary engine (not Spark-based*)
- Fully managed — you don't see infrastructure
- Virtual warehouses auto-scale, auto-suspend
- Less control, less to configure
- SQL-first, expanding to Python via Snowpark
- Stronger for: SQL analytics, BI, data sharing

Origins still shape how they work. The team you have determines which is easier to adopt.

Infrastructure Models Compared

Databricks

Control Plane ↔ Data Plane

- You control the data plane
- Can inspect compute resources
- More governance flexibility
- More operational responsibility

Snowflake

Fully Managed → Virtual Warehouses

- Snowflake manages everything
- You just choose warehouse size (XS–6XL)
- Simpler operations
- Less visibility into underlying infrastructure

When to Use Which

Choose Databricks When...

- Heavy data engineering + ML pipelines
- Team is Python/Spark-proficient
- Need streaming + batch on one platform
- Want data plane in your own cloud account

Choose Snowflake When...

- SQL-first analytics and BI
- Team is analyst-heavy
- Want minimal operational overhead
- Need secure cross-org data sharing

Use Both When...

- Large org with mixed workloads
- Engineering team + analytics team
- Databricks for pipelines → Snowflake for BI
- 52% of Snowflake customers do this

The binding constraint is your team's skills and your organization's workload mix — not which platform is 'better.'

Notebooks & the Compute Model

Where You Write Code and Where It Runs

Multi-Language Notebooks: Now Industry Standard

Databricks Notebook

```
# Python cell (default)
df =
spark.read.table("cat_cafe.visits")
display(df.limit(5))
```

```
%sql
SELECT cat_id, COUNT(*) as visits
FROM cat_cafe.visits
GROUP BY cat_id
```

Snowflake Notebook

```
# Python cell (Snowpark)
session = get_active_session()
df =
session.table("cat_cafe.visits")
df.show(5)
```

```
-- SQL cell
SELECT cat_id, COUNT(*) as visits
FROM cat_cafe.visits
GROUP BY cat_id
```

Databricks pioneered multi-language notebooks. Snowflake adopted the same pattern.

Compute Models: A Spectrum of Control

← More control

More convenience →

**Self-managed
EC2**

*Maximum control,
maximum responsibility*

**Databricks
clusters**

*You choose instance
types, cluster size*

**Databricks
serverless**

*Databricks manages
infra. You write code.*

**Snowflake
virtual warehouses**

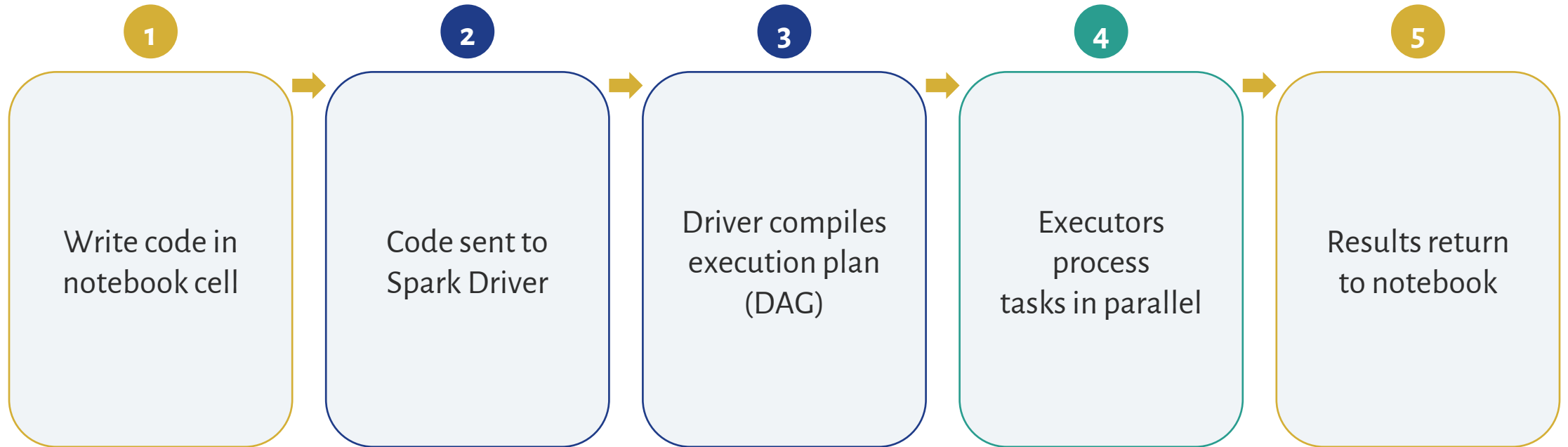
*Choose size (XS–6XL).
Auto-suspend.*

**Snowflake
serverless tasks**

*Fully managed.
Pay per query.*

The same control-vs-convenience tradeoff as Lambda vs. EC2, or Athena vs. self-managed Presto

What Happens When You Click 'Run'



The notebook is the interface. Spark is the engine. The cluster is the infrastructure.

Databricks Cluster Types

All-Purpose (Interactive)

- Stays running until manually terminated
- Supports multiple users
- Used for: exploration, development
- Higher cost — runs even when idle

Job Cluster (Ephemeral)

- Created for one task, terminated after
- Single-purpose execution
- Used for: scheduled ETL, production
- Lower cost — runs only during the job

Cost warning: A cluster left running overnight at \$2/hr = \$16 wasted. A month = \$480. Always terminate when done.

Session Objects: Your API Entry Point

Databricks (SparkSession)

```
# Pre-configured in Databricks
spark # already available

# Read a table
df =
spark.read.table("cat_cafe.visits")
df.filter(df.duration_minutes >
30).show()
```

Snowflake (Snowpark Session)

```
# Get session
session = get_active_session()

# Read a table
df =
session.table("cat_cafe.visits")
df.filter(df.duration_minutes >
30).show()
```

Same pattern: create a session object, use it to read tables. Like Boto3 sessions for AWS.

SQL Across Platforms

PostgreSQL vs. Spark SQL vs. Snowflake SQL

Your SQL Knowledge Transfers

90% of the SQL you wrote in Weeks 3–5 works identically across PostgreSQL, Spark SQL, Snowflake, and Databricks SQL.

```
SELECT
  c.name AS cat_name,
  COUNT(v.visit_id) AS total_visits,
  AVG(v.duration_minutes) AS avg_duration
FROM cat_cafe.visits v
JOIN cat_cafe.cats c ON v.cat_id = c.cat_id
WHERE v.visit_date ≥ '2025-01-01'
GROUP BY c.name
ORDER BY total_visits DESC;
```

ANSI SQL Compliant: all platforms

Platform-Specific Syntax (Minor Differences)

Task	PostgreSQL	Spark SQL / Snowflake
Inspect table	<code>\d table_name</code>	<code>DESCRIBE table_name</code>
List tables	<code>\dt</code>	<code>SHOW TABLES</code>
Temp table	<code>CREATE TEMP TABLE</code>	<code>CREATE TEMP VIEW</code>
Time travel	N/A	<code>VERSION AS OF / AT(TIMESTAMP...)</code>

Syntax differences are minor. The execution model difference is what matters.

Same SQL, Different Physics

PostgreSQL



Single Server

- Single-node query planner
- GROUP BY runs on one core
- Bound by one machine

Spark SQL / Snowflake



Distributed Cluster

- Distributed query compiler
- GROUP BY becomes a shuffle across the cluster
- Scales with cluster/warehouse size

The SQL is the same. The physics underneath is completely different.

Week 9 → Week 10 Connection

*In Week 9, you learned that GROUP BY at massive scale requires a shuffle.
In Week 10, you write that GROUP BY in Databricks or Snowflake and the platform does the shuffle automatically.*

The hard problem is hidden. It's not eliminated.

Week 11: We'll look at what that shuffle actually costs and how to optimize it.

Cloud-to-Cloud ETL

Moving from Local Compute to Cloud Compute

Three Weeks, Three Compute Patterns

Week 9 : Cloud Data, Local Compute

RDS PostgreSQL → download → Your Laptop (pandas/sklearn)

Binding constraint: laptop memory and CPU

Week 10: Cloud Data, Cloud Compute

RDS / S3 → staging → Snowflake / Databricks

Binding constraint: platform config and cost

Week 12: Cloud Data, Self-Managed Compute

S3 → EMR (your Spark cluster)

Binding constraint: infrastructure knowledge

Same data, same questions. The compute location changes. The binding constraint shifts.

JDBC Direct Read (PySpark in Databricks)

RDS PostgreSQL



JDBC



Databricks Notebook (Spark)

```
df = (spark.read
      .format("jdbc")
      .option("url",
              "jdbc:postgresql://rds:5432/catcafe")
      .option("dbtable", "cat_cafe.visits")
      .option("user", "student")
      .option("password",
              dbutils.secrets.get("rds", "password"))
      .option("driver",
              "org.postgresql.Driver")
      .load())
```

```
df.groupBy("cat_id").count().show()
```

Works on Databricks Free Edition

Never hardcode passwords. Use dbutils.secrets.

Databricks Free Edition: What You Can and Can't Do

Available ✓

- Serverless compute (notebooks)
- Unity Catalog (basic)
- Delta Lake tables
- SQL, Python, Scala, R
- Sample datasets
- `spark.read.format('jdbc')`

Not Available ✗

- Lakehouse Federation
- Classic clusters (configurable)
- Job scheduling
- Custom libraries via init scripts
- Most partner integrations

Free Edition is a learning sandbox. Concepts and code patterns transfer to production.

In-Class Demo: Snowflake Cloud-to-Cloud ETL

1

Create External Integration

Snowflake → AWS IAM role → S3 access grant

2

Create External Stage

Points to s3://ppol5206-catcafe/ with CSV format

3

COPY INTO Target Table

Snowflake warehouse reads S3, loads into Snowflake table

4

Validate and Query

Run the same SQL from Weeks 3–5 on cloud-hosted data

Live demo — follow along in Snowflake trial account

Snowflake: The Same Query, Cloud Compute

S3 Bucket (CSV)



Snowflake Table



SQL Worksheet (Query)

```
SELECT
  c.name,
  COUNT(v.visit_id) AS visits,
  AVG(v.duration_minutes) AS avg_duration
FROM cat_cafe.visits v
JOIN cat_cafe.cats c
  ON v.cat_id = c.cat_id
GROUP BY c.name
ORDER BY visits DESC;
```

Exact same query from Project 2. Now running on cloud compute.

Three Platforms, One Question

How do I get data from source A into platform B for analysis?

Local (Wk 5-8)

psycopg2 + pandas
download to local
Single-thread

Snowflake (Wk 10)

COPY INTO from S3
Cloud-to-cloud
Warehouse compute

Databricks (Wk 10)

spark.read.jdbc
Cloud-to-cloud
Spark cluster compute

Take-Home Lab: Databricks Cloud-to-Cloud ETL

Lab Steps

- Sign up for Databricks Community Edition
- Upload the provided CSV to DBFS or use JDBC
- Create a notebook: Python + SQL cells
- Load data into a Delta Lake table
- Run 3 analytical queries from Project 2
- Compare execution to local PostgreSQL

Deliverables

- Screenshot: Notebook with code + results
- Screenshot: Unity Catalog table created
- Short reflection: what was easier/harder vs. PostgreSQL local?